

# PHOENIX – API Data Exchange Specification

|              |                                                               |
|--------------|---------------------------------------------------------------|
| Prepared by: | Vipul                                                         |
| Role:        | Cybersecurity Team Member                                     |
| Project:     | PHOENIX                                                       |
| Deliverable: | Threat Analysis Integration - API Data Exchange Specification |

## 1. Purpose

This document sets out the official technical specification for data exchange across the PHOENIX Threat Analysis Integration. It clearly defines how external event sources, the Threat Analysis Engine, the ADCRS Risk Scoring Engine, the Response Decision Manager, and the Output Layer communicate with one another.

The goal is to ensure that all data payloads, request parameters, response structures, and security controls are strictly standardised so that our services can integrate smoothly, reliably, and securely without data mismatch.

## 2. Architecture, Scope & Team Traceability

This specification directly implements and operationalises the architecture established by our team deliverables:

| Integration Component / Layer         | Core Document Reference                      | Lead Author   | Alignment in this Spec                               |
|---------------------------------------|----------------------------------------------|---------------|------------------------------------------------------|
| Integration Scope & Security Controls | Threat Analysis Integration Design Document  | Mannat        | Security controls (Sec 8) & Common event definitions |
| Component Architecture & Data Flow    | Threat Analysis Integration Workflow Diagram | Dilano Xavier | Endpoints 4.1 - 4.4 routing & 7-layer data path      |
| Chronological Message Flow            | Threat Analysis Sequence Diagram             | Dilano Xavier | Request/Response cycles & status codes (Sec 1-8)     |
| Test Cases & Verification Scenarios   | Threat Analysis Integration Testing Plan     | Dilano Xavier | Payload validation for FIT-01..14 & SIT-01..09       |

## 3. Global Communication Standards & Security

[Source Alignment: Mannat & Dilano – Design Document Sec. 8 & Workflow Diagram Sec. 3]

All integration endpoints must strictly adhere to the baseline security controls agreed in the team design:

- Protocol & Transport:** All communication must use HTTPS (TLS 1.3) to guarantee confidentiality and integrity in transit.
- Data Format:** All request bodies and responses must use standard JSON (`application/json`) encoded in UTF-8.

- **Authentication (JWT):** Every protected API request must include a valid JSON Web Token in the HTTP Authorization header: `Authorization: Bearer <token>`.
- **Access Control (RBAC):** Access is restricted according to service and user roles (e.g., `EventProducer`, `ThreatAnalyst`, `SecurityAdmin`). Requests with insufficient privileges are immediately denied.
- **Rate Limiting:** Endpoints enforce request limits (e.g., 100 requests/minute per client IP/token) to prevent DDoS and API abuse.
- **Input Validation:** Strict schema validation is enforced before any processing. Requests containing unexpected types, missing mandatory fields, or malformed JSON are rejected with HTTP 400.

## 4. API Endpoints Specification

### 4.1 Ingest Security Event

[Source Alignment: Mannat & Dilano – Design Doc Sec. 6 & Testing Plan Sec. 4 (Input Structure: EVT-001)]

**Description:** Allows input sources (TEAVS alerts, application logs, IoT events, etc.) to submit new security events into the Event Collection Layer for validation and threat analysis.

- **Method & URI:** POST /api/v1/events/ingest
- **Access Role Required:** EventProducer, SecurityAdmin
- **Success Status Code:** 202 Accepted (asynchronous processing) or 200 OK

Request Headers:

| Header Name     | Type   | Required | Description                                                    |
|-----------------|--------|----------|----------------------------------------------------------------|
| Authorization   | String | Yes      | Bearer JWT access token (Security Control: JWT Authentication) |
| Content-Type    | String | Yes      | application/json                                               |
| X-Source-System | String | Yes      | Identifier of the producing system (e.g., TEAVS, IoT-Gateway)  |

Request Payload Schema (Derived from Mannat's Design Doc Sec. 6 & Dilano's Testing Plan Sec. 4):

```
{
  "event_id": "EVT-001",
  "event_type": "login_activity",
  "timestamp": "2026-08-03T06:30:00Z",
  "source": "TEAVS",
  "event_data": {
    "user_id": "usr_94812",
    "ip_address": "192.168.1.105",
    "failed_attempts": 5,
    "user_agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
  }
}
```

Field Definitions:

| Field      | Type              | Required | Description                                                            |
|------------|-------------------|----------|------------------------------------------------------------------------|
| event_id   | String            | Yes      | Unique identifier for the event (e.g., 'EVT-001')                      |
| event_type | String            | Yes      | Type of event (e.g., 'login_activity', 'api_request', 'iot_telemetry') |
| timestamp  | String (ISO 8601) | Yes      | Event generation timestamp in UTC format                               |
| source     | String            | Yes      | Name of the source component (e.g., 'TEAVS', 'API_Gateway', 'IoT')     |
| event_data | Object            | Yes      | Contextual metadata specific to the event type                         |

Success Response (202 Accepted):

```
{
  "status": "success",
  "message": "Event received and queued for threat analysis.",
  "event_id": "EVT-001",
  "received_at": "2026-08-03T06:30:01Z"
}
```

4.2 Evaluate Threat Risk Score (ADCRS Interface)

[Source Alignment: Dilano Xavier – Sequence Diagram Step 5 (ADCRS Risk Scoring) & Steps 10-10.1]

**Description:** Internal service interface used by the Threat Analysis Engine to forward normalised event parameters and rule-detection results to the ADCRS AI Engine for risk scoring.

- **Method & URI:** POST /api/v1/adcrs/evaluate
- **Access Role Required:** ThreatAnalysisService, InternalSystem
- **Success Status Code:** 200 OK

Request Payload Schema:

```
{
  "event_id": "EVT-001",
  "event_type": "login_activity",
  "timestamp": "2026-08-03T06:30:00Z",
  "source": "TEAVS",
  "rule_detection_result": {
    "rule_id": "RULE-AUTH-04",
    "threat_category": "Login Attack",
    "preliminary_severity": "High"
  }
}
```

```

    },
    "context_attributes": {
      "failed_attempts": 5,
      "geographic_anomaly": true,
      "ip_reputation_score": 78
    }
  }
}

```

Success Response (200 OK - ADCRS Calculation Output):

```

{
  "status": "success",
  "event_id": "EVT-001",
  "risk_score": 82,
  "confidence_score": 0.91,
  "threat_classification": "Login Attack",
  "evaluation_timestamp": "2026-08-03T06:30:02Z"
}

```

### 4.3 Dispatch Final Threat Analysis Result

[Source Alignment: Mannat & Dilano – Design Doc Sec. 7 & Testing Plan Sec. 5 & 9 (Output Structure)]

**Description:** Transfers the synthesized decision output from the Response Decision Manager to the Output Layer for alerting, audit logging, and persistence.

- **Method & URI:** POST /api/v1/threats/results
- **Access Role Required:** DecisionManagerService, SecurityAdmin
- **Success Status Code:** 201 Created

Standard Output Payload Schema (Aligned with Mannat's Design Doc Sec. 7 & Dilano's E2E-01 Output):

```

{
  "event_id": "EVT-001",
  "threat_type": "Login Attack",
  "severity": "High",
  "risk_score": 82,
  "confidence_score": 0.91,
  "recommended_action": "lock_account",
  "status": "detected",
  "decision_details": {
    "notification_required": true,
    "dashboard_alert": true,
    "audit_logged": true
  }
}

```

### 4.4 Fetch Threat Records for Dashboard

[Source Alignment: Dilano Xavier – Workflow Diagram Step 7 (Dashboard Updates) & Testing Plan FIT-10]

**Description:** Enables the PHOENIX Security Dashboard to query processed threat events with filtering support.

- **Method & URI:** GET /api/v1/dashboard/threats
- **Access Role Required:** ThreatAnalyst, SecurityAdmin, DashboardViewer
- **Query Parameters:** `severity` (optional), `status` (optional), `limit` (default: 50), `offset` (default: 0)
- **Success Status Code:** 200 OK

Success Response (200 OK):

```
{
  "total_records": 1,
  "limit": 50,
  "offset": 0,
  "data": [
    {
      "event_id": "EVT-001",
      "threat_type": "Login Attack",
      "severity": "High",
      "risk_score": 82,
      "confidence_score": 0.91,
      "recommended_action": "lock_account",
      "status": "detected",
      "timestamp": "2026-08-03T06:30:02Z"
    }
  ]
}
```

## 5. Secure Error Handling & Status Codes

[Source Alignment: Dilano Xavier – Testing Plan SIT-04 (Input Validation) & SIT-06 (Secure Error Handling)]

To prevent internal system details, stack traces, or database structures from leaking to potential attackers, all error responses follow a strict, uniform JSON format.

Standard Error Response Format:

```
{
  "error": {
    "code": "INVALID_PAYLOAD",
    "message": "The request body failed schema validation. 'event_id' is mandatory.",
    "status_code": 400,
    "timestamp": "2026-08-03T06:30:03Z",
    "tracking_id": "ERR-9921-X9"
  }
}
```

Summary of HTTP Status Codes:

| HTTP Code | Status Text | Condition / Meaning |
|-----------|-------------|---------------------|
|-----------|-------------|---------------------|

|     |                       |                                                                             |
|-----|-----------------------|-----------------------------------------------------------------------------|
| 200 | OK                    | Request processed successfully and synchronous data returned.               |
| 201 | Created               | New threat record successfully created and stored.                          |
| 202 | Accepted              | Event received, validated, and placed in processing queue.                  |
| 400 | Bad Request           | Missing required fields, malformed JSON, or schema validation failure.      |
| 401 | Unauthorized          | Missing, invalid, or expired JWT Bearer token.                              |
| 403 | Forbidden             | Authenticated entity lacks required RBAC role or permissions.               |
| 404 | Not Found             | Target threat record, event ID, or endpoint does not exist.                 |
| 429 | Too Many Requests     | Rate limit exceeded; request rejected to protect service stability.         |
| 500 | Internal Server Error | Unexpected processing error handled securely without exposing debug traces. |

## 6. Summary & Traceability

This API Data Exchange Specification bridges the gap between high-level architectural models and actual backend implementation. It provides exact schema definitions, endpoint contracts, and security enforcement criteria that directly support:

- **Integration Testing:** Supplies exact payload structures for executing test cases FIT-01 through FIT-14 and SIT-01 through SIT-09 (prepared by Dilano).
- **Component Alignment:** Ensures TEAVS, ADCRS, the backend rules engine, and the dashboard interact seamlessly without schema conflicts based on Mannat's design.
- **Cybersecurity Compliance:** Validates core security controls including JWT authentication, RBAC, HTTPS, rate limiting, and sanitized error handling across all communication channels.